PART 11

Performance & Networking

88 Cost Optimization
89 Cold Starts
90 Serverless Limits
91 Latency
92 Throughput
93 P99 Latency
94 Tail Latency
95 Network Partitions
96 Clock Skew
97 DNS
98 TCP vs UDP
99 HTTP/2 & HTTP/3
100 gRPC

#088

Cost Optimization

WHAT IT IS

Getting the same outcomes for less cloud spend.

WHY IT MATTERS

Infrastructure bills grow quietly and can sink a SaaS margin.

EXAMPLE

Right-size instances, use autoscaling, buy reserved/savings plans for steady load, spot instances for queue workers, S3 lifecycle rules (move old files to cold storage), cap log retention, cache to avoid DB upgrades, tag resources per team/tenant to see who spends what, and set budget alerts.

#089

Cold Starts

WHAT IT IS

The delay when a new instance (serverless function, new pod, cold cache) handles its first request.

WHY IT MATTERS

Causes slow first responses, especially after scale-to-zero.

EXAMPLE

Lambda/Vapor PHP function boot adds hundreds of ms. Reduce: smaller package, php artisan config:cache route:cache event:cache, OPcache preloading, provisioned concurrency, warm-up requests, and Octane to keep the app booted.

#090

Serverless Limits

WHAT IT IS

Hard constraints of serverless platforms: max execution time, memory, payload size, concurrency, ephemeral disk.

WHY IT MATTERS

Designs that ignore them fail at scale or cost a lot.

EXAMPLE

~15 min max runtime, 6 MB sync payload, /tmp only, concurrency cap per account, no long-lived connections (use RDS Proxy for DB). Long imports → break into queued chunks; big uploads → pre-signed S3 URLs; WebSockets → managed service.

#091

Latency

WHAT IT IS

Time for one request to complete (or one hop to travel).

WHY IT MATTERS

Directly affects user experience and conversion.

EXAMPLE

```
$responses = Http::pool(fn (Pool $p) => [$p->get($a),
    $p->get($b), $p->get($c)]); // concurrent
```

#092

Throughput

WHAT IT IS

Work completed per unit time (requests/second, jobs/minute).

WHY IT MATTERS

Capacity planning; related to latency but not the same (batching can raise throughput and latency).

EXAMPLE

```
k6 run --vus 50 --duration 60s loadtest.js
```

#093

P99 Latency

WHAT IT IS

The 99th percentile: 99% of requests are faster than this; 1% are slower.

WHY IT MATTERS

Averages hide pain. A 100ms average can coexist with a 3s P99.

EXAMPLE

Report P50/P95/P99 per endpoint. If P99 = 2.4s on /dashboard, find the slow 1% (large tenants? cold cache? lock waits?) with traces and histograms.

#094

Tail Latency

WHAT IT IS

The slowest slice of requests (P99, P99.9).

WHY IT MATTERS

In fan-out systems, one slow call among many makes the whole request slow, so tail dominates real experience.

EXAMPLE

A page calling 10 services, each with 1% slow chance, is slow ~10% of the time. Mitigate with timeouts, hedged requests, caching, reducing fan-out, avoiding GC/lock pauses, and load shedding.

#095

Network Partitions

WHAT IT IS

Network failure splits a system into groups that cannot talk to each other (though each may be healthy).

WHY IT MATTERS

Causes split-brain and inconsistent data; it is the "P" in CAP.

EXAMPLE

Two DB nodes both think they are primary after a link failure. Prevent with quorum (majority required, e.g. 2 of 3), fencing, and leader leases. In app code: timeouts, retries with idempotency, and graceful degradation.

#096

Clock Skew

WHAT IT IS

Different machines' clocks disagree.

WHY IT MATTERS

Breaks ordering, token expiry, locks, cache TTLs, and "last write wins".

EXAMPLE

```
JWT::$leeway = 30; // seconds
```

#097

DNS

WHAT IT IS

The system that turns names (app.example.com) into IP addresses.

WHY IT MATTERS

Misconfigured DNS or TTLs cause outages and slow failovers.

EXAMPLE

```
dig +short app.example.com
dig app.example.com @1.1.1.1 +trace
```

#098

TCP vs UDP

WHAT IT IS

TCP = reliable, ordered, connection-based. UDP = fast, connectionless, no guarantees.

WHY IT MATTERS

Choose by need: correctness vs. latency.

EXAMPLE

```
$sock = stream_socket_client('udp://127.0.0.1:8125');
fwrite($sock, "taskflow.jobs.processed:1|c"); //
    StatsD metric: loss is acceptable
```

#099

HTTP/2 & HTTP/3

WHAT IT IS

HTTP/2 multiplexes many requests over one TCP connection (header compression, server push dropped). HTTP/3 runs on QUIC (UDP) avoiding TCP head-of-line blocking and speeding up connection setup.

WHY IT MATTERS

Faster page loads, especially on mobile/lossy networks.

EXAMPLE

```
listen 443 ssl;
http2 on;
listen 443 quic reuseport;       # HTTP/3
add_header Alt-Svc 'h3=":443"; ma=86400';
```

#100

gRPC

WHAT IT IS

A high-performance RPC framework using HTTP/2 and Protocol Buffers (binary, typed contracts, streaming).

WHY IT MATTERS

Efficient service-to-service calls with generated clients.

EXAMPLE

```
service Billing { rpc CreateInvoice (InvoiceRequest)
    returns (InvoiceReply); }
message InvoiceRequest { string customer_id = 1;
    int64 amount_cents = 2; }
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).